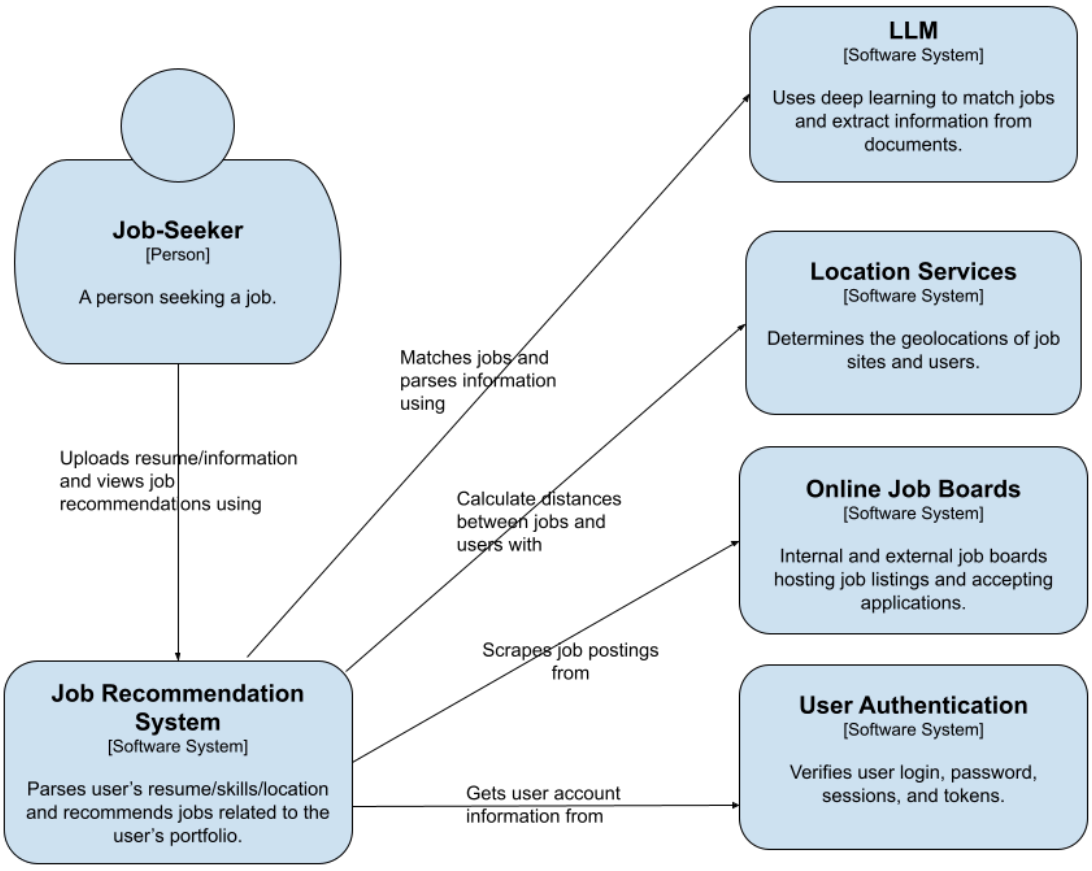
ECE 49595 SD I – Design Document

Team 1: JobReccers

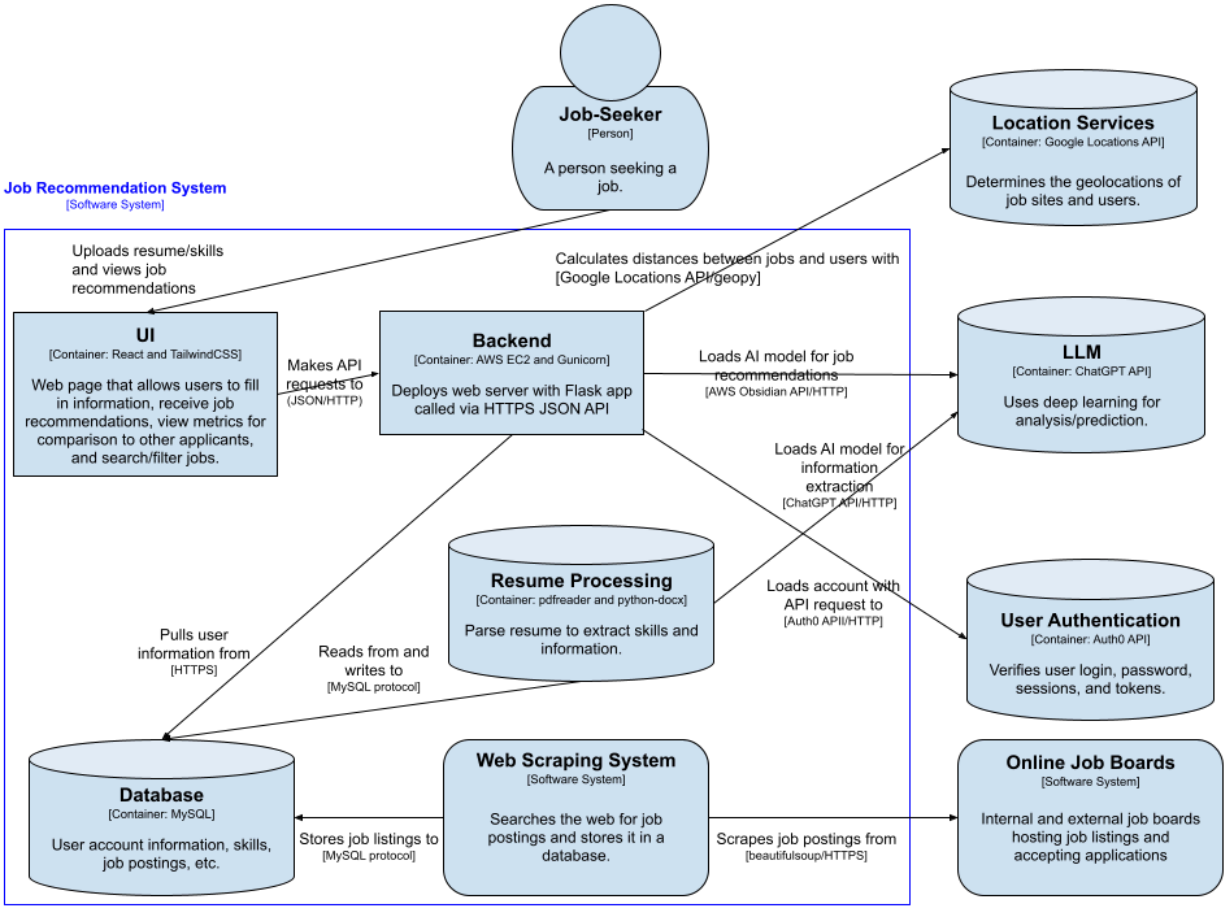
May 2nd, 2026

Design Diagrams

System Context



Container


Job-Seeker
[Person]
A person seeking a job.
Location Services
[Container: Google Locations API]
Determines the geolocations of job sites and users.
Job Recommendation System
[Software System]
Uploads resume/skills and views job recommendations
Calculates distances between jobs and users with [Google Locations API/geopy]
UI
[Container: React and TailwindCSS]
Web page that allows users to fill in information, receive job recommendations, view metrics for comparison to other applicants, and search/filter jobs.
Makes API requests to
(JSON/HTTP)
Backend
[Container: AWS EC2 and Gunicorn]
Deploys web server with Flask app called via HTTPS JSON API
Loads AI model for job recommendations
[AWS Obsidian API/HTTP]
LLM
[Container: ChatGPT API]
Uses deep learning for analysis/prediction.
Loads AI model for information extraction
[ChatGPT API/HTTP]
Resume Processing
[Container: pdfreader and python-docx]
Parse resume to extract skills and information.
Loads account with API request to
[Auth0 API/HTTP]
User Authentication
[Container: Auth0 API]
Verifies user login, password, sessions, and tokens.
Pulls user information from
[HTTPS]
Reads from and writes to
[MySQL protocol]
Database
[Container: MySQL]
User account information, skills, job postings, etc.
Web Scraping System
[Software System]
Searches the web for job postings and stores it in a database.
Stores job listings to
[MySQL protocol]
Scrapes job postings from
[beautifulsoup/HTTPS]
Online Job Boards
[Software System]
Internal and external job boards hosting job listings and accepting applications

Tech Stack

Backbone

- **Programming language:** We will be using Python, as it is what most of the team has experience with and has the most tooling for accessing LLMs.
- **Server deployment:** We will deploy the website on AWS EC2 (Elastic Compute Cloud), which allows us to run a linux virtual machine on AWS servers and assign it a public IP. We are using this because we do not have access to our own server and have experience with the AWS ecosystem.
- **Database:** The used database will be MySQL, which is a stable SQL implementation maintained by Oracle. We are using this over NoSQL options because we require a relational model for structuring our data between users, skills, and job listings.
- **ORM:** We will use SQLAlchemy to serialize our job listing and user types into the MySQL database. This is a more robust approach to storing and retrieving python objects than manually creating and updating table schemas when changes are made to our data types, which is especially beneficial for security-critical portions of the project like user authentication.
- **User Authentication:** We will use Auth0 for user authentication (sign-up/login, password resets). Auth0 implements industry-standard protocols like OAuth and will issue signed JWTs that our backend can verify to protect routes and associate requests with a user. This reduces the risk of implementing authentication incorrectly and lets us centralize identity/security concerns.
- **Webserver:** The server code will be written in Flask because it is a simple python framework for hosting web servers. Gunicorn, a production WSGI server, will run the application in the production environment because the built-in Flask runner is only intended for development purposes. Finally, this will run behind nginx as a reverse proxy so we can have features such as rate limiting and performant static file hosting.

Testing frameworks

- **Jest** is a JavaScript testing framework, which will be used to validate our frontend code for correctness of functionality.
- **Pytest** is a python testing framework which will be used to validate our backend code for correctness through unit and integration tests.
- **Selenium** is a browser-simulating GUI tester which will be used to validate the functionality of the webui from a user perspective.

Resume reading

- **PDF:** We will use the python package pdfreader to read PDF resumes because it is the only package on PyPI for this purpose.
- **Word:** We will use python-docx to read resumes as word documents because it is the only package on PyPI for this purpose.

Document Parsing

- **LLM:** We will use ChatGPT's API. This is a system that the team has the most familiarity with, but is subject to change as we experiment and determine which models produce the best results through trial and error.

User Interface

- **Frontend framework:** We will use React for creating the frontend because it is the JavaScript framework we have the most experience with.

Recommendations

- **Geolocation:** We will use the google locations API to determine the geolocations of job sites and users, then use the python package geopy to calculate the distance between the two to provide more relevant job listings. Geopy is a general purpose python library for geographical calculations, making it suitable for our purposes.

Job Listing Aggregation

- **Requests:** The built-in requests package will be used to retrieve listing information from publicly-accessible APIs
- **Web Scraping:** When the builtin requests package doesn't suffice for a task and we need to scrape webpages, we will use the python package beautifulsoup since it provides a structured way to navigate HTML documents. We will also use playwright to render dynamic pages when basic HTML parsing does not suffice.

Figma

A view-only copy of our Figma design can be found here.

Design Trades

Skill Detection Subsystem

Designs

D-001 Static Skill Repository with Word Matcher: A list of skills is generated by taking a representative sample of job listings as they appear today, then using humans and/or LLMs to parse out skill names from them. These skills are then grouped into synonym groups and assigned a canonical, representative member for the sake of storage. The synonym group is stored for reference. Then, when the system is given a resume or job listing to pull skill information from, it searches the document for any words contained in the canonical skill list or their synonyms and assigns the canonical names for these words as skills.

D-002 Snapshotted Skill Repository with LLM Matcher: A list of skills is generated by taking a representative sample of job listings as they appear today, then using LLMs to parse out skill names from them. These skills are then grouped into synonym groups and assigned a canonical, representative member for the sake of storage. Only the canonical skill name is stored, and each skill is given a broad category for searching purposes. On a monthly basis, this process is

repeated using job listings stored in the database. Job listings used for this purpose are then marked so that they are not included in this process again.

D-003 Dynamic Skill Repository with LLM Matcher: A list of skills is generated by taking a representative sample of job listings as they appear today, then using LLMs to parse out skill names from them. These skills are then grouped into synonym groups and assigned a canonical, representative member for the sake of storage. Only the canonical skill name is stored, and each skill is given a broad category for searching purposes. On a monthly basis, this process is repeated using job listings stored in the database. Job listings used for this purpose are then marked so that they are not included in this process again.

Evaluation

Evaluation Rankings (out of 10)

Design	False Positive Resilience	False Negative Resilience	Maintainability	Level of Detail	Cost Efficiency	Performance
D-001	10	3	1	4	10	10
D-002	5	8	10	10	3	3
D-003	5	10	10	10	1	1

Final Scores:

Design	Total (out of 10)
D-001	5.7
D-002	6.7
D-003	7.4

Decision

In the end, we have decided to use the dynamic skill repository with LLM matcher. While it is the least cost-performant and most time-consuming option available, it also provides the best accuracy while providing the greatest level of detail and requiring the least human involvement during maintenance. While some aspects of its design could lead to false negatives, we believe that it is an overall net gain compared to D-002 and can be mostly remedied through automated, periodic processes. While the snapshotted skill repository is still preferable to a static one for maintainability reasons, it does not substantially improve on accuracy, which is why D-003 is more preferable.

Design Trade #2:

Design Alternatives Summary

- **Design A: Static/API-First Scraper with Source-Specific Parsers**
  This design uses APIs where available and static HTML parsing with source-specific parsers to extract job data, prioritizing simplicity and low infrastructure overhead.
- **Design B: Headless Browser Scraper**
  This design uses browser automation (e.g., Playwright or Selenium) to render and scrape dynamic, JavaScript-heavy job pages for improved content coverage.
- **Design C: Hybrid Multi-Source Pipeline (Selected)**
  This design combines API-first collection, static scraping, and headless browser fallback to balance performance, scalability, and coverage across diverse job sources.

Criterion	Weight	Design A	Weighted	Design B	Weighted	Design C	Weighted
Extraction Accuracy	30	7	210	8	240	9	270
Link Reliability / Freshness	20	7	140	7	140	9	180
Scalability / Performance	20	9	180	5	100	8	160
Maintainability / Extensibility	15	7	105	6	90	8	120

Location Data Quality	15	7	105	8	120	9	135
Total	**100**		**740**		**690**		**865**

The selected solution is **Design C: Hybrid Multi-Source Pipeline**, as it achieved the highest overall score and provides the best balance across all critical evaluation criteria. It improves extraction accuracy and coverage compared to Design A by supporting dynamic content, while avoiding the heavy performance and scalability costs of Design B's browser-only approach. Although it introduces additional complexity in routing logic and system design, this trade-off is justified by its flexibility and robustness across diverse job sources. We prioritized long-term scalability, maintainability, and data reliability over simplicity, making this design the most suitable for a production-ready job recommendation system.

Design Trade #3:

D-001: Rule Based Ranking System: This system ranks job utilized structured inputs that have been provided by other subsystems which extracted user skills, job skills, and location data. Through this data, a score is computed that is based on features such as skill overlap, location overlap. This score is manually defined, based on weights that are defined by the programmer. Jobs are then ranked by this numerical score, and the highest scores are provided to users as job recommendations.

D-002: Feature-Based ML Ranking System: This design utilizes structured features which have already been provided by other subsystems which extracted user skills, job skills, and location data. Rather than manually determining fixed weights of each factor however, this data is inputted into an ML model, such as a logistic regression model, which learns what factors are the most important based on training data, and computes a relevance score. Jobs are then ranked by this numerical score, and the highest scores are provided to users as job recommendations.

D-003: Hybrid Semantic and Feature Based Ranking System: This design utilizes structured features which have already been provided by other subsystems which have already extracted data on user skills, job skills, and location data. Rather than relying only on structured features or a single model, this design combines structured feature-based scoring and semantic similarity. Along with features such as skill overlap and location match, the system also computes a semantic similarity score that is able to take into account deeper relationships between user profiles and job requirements. These components are then combined to compute a final super score. Jobs are then ranked by this numerical score calculated, and the highest scores are provided to users as job recommendations.

Trade Study Matrix

First value in table is out of 10, second is value based on percent weight given to column:

Design	Accurac	Explanat	Respon	Scalabili	Persona	Comple	Total

	y (30%)	ion (20%)	se Time (15%)	ty (15%)	lization (10%)	xity (10%)	Weighte d Score
Rule Based Ranking System	5 (1.50)	9 (1.80)	10 (1.50)	8 (1.20)	5 (0.50)	9 (0.90)	7.40
Feature-Based ML Ranking System	7 (2.10)	8 (1.60)	8 (1.20)	8 (1.20)	6 (0.60)	7 (0.70)	7.40
Hybrid Semantic and Feature Based Ranking System	9 (2.70)	8 (1.60)	7 (1.05)	9 (1.35)	9 (0.90)	5 (0.50)	8.10

The hybrid design of having both a semantic and structured feature based ranking model (Which was the third design in the trade study matrix above) was selected as it achieved the highest overall score based on accuracy, explanation, response time, scalability, personalization, and complexity factors. Through combining both semantic similarity with features such as skill overlap and location preferences, job recommendations given to users are more accurate. One setback, however, to this design is that it is more complex than the other designs. However, this complexity makes it better for supporting system features such as transparent and accurate explanations as to why a job has been selected for a user.